The Parallel Science Project: Cyber Space for Human–AI Co-Evolution of Science

Authors

April 8, 2026

Abstract

We introduce Parallel Science, an open infrastructure that establishes a dedicated scientific publication space for AI-generated research—a "parallel arXiv" that runs alongside and interacts with the human scientific ecosystem. At its core is papers.parallelscience.org, a preprint repository where AI research systems continuously publish versioned, categorized papers with stable identifiers, full provenance, and open access. The publication infrastructure is agnostic to the upstream research system: any autonomous pipeline that produces papers and pushes them to GitHub can publish to the repository. Three systems are currently connected—a fleet of Denario research agents operating under human supervision, a fleet operating autonomously, and the CosmoEvolve Virtual Lab—spanning topics from fluid dynamics to observational cosmology. We describe the publication infrastructure, the fleet architecture, the end-to-end data flow from research task to indexed preprint, and present examples of published papers. The system is operational and producing papers at a cost of under one dollar per paper, creating a growing body of machine-generated scientific literature that is openly accessible, citable, and distinct from human-authored work.

1 Introduction

The scientific literature is a human institution. For over three centuries, researchers have communicated discoveries through manuscripts written by humans, reviewed by humans, and read by humans. The arXiv preprint server, launched in 1991 [Ginsparg, 2011], accelerated this process by making papers freely and immediately available, but the fundamental model remains unchanged: humans produce scientific knowledge, and the literature is the record of that production.

Large language models and multi-agent systems are now capable of generating scientific hypotheses, writing and executing code, analyzing data, and producing coherent research papers [Lu et al., 2024, Boiko et al., 2023, Huang et al., 2024]. This raises a question that is as much institutional as it is technical: *where should machine-generated scientific work be published?*

We argue that AI-generated research should not be mixed indiscriminately into the existing human literature. The provenance is different, the failure modes are different, and the standards of evaluation are still being established. At the same time, AI-generated results should not be hidden or discarded—they may contain genuine insights, and transparency about what machines produce is essential for the co-evolution of human and artificial intelligence in science.

Parallel Science proposes a resolution: a *parallel* scientific publication space that mirrors the structure of the human literature—with preprints, stable identifiers, categories, versioning, and open access—but is explicitly designated as the output of AI research systems. This parallel space operates alongside the human arXiv, not as a replacement but as a complement. Human scientists

can browse, cite, and build upon AI-generated work; AI systems can, in turn, access and build upon human-authored papers. The two spaces evolve in parallel, with controlled interaction.

The concrete realization of this idea is papers.parallelscience.org, a preprint repository that we call **Parallel ArXiv**. Crucially, the publication infrastructure is agnostic to the upstream research system: it accepts papers from any autonomous pipeline that publishes to GitHub Pages with the expected metadata format. Three such systems are currently connected, spanning supervised and unsupervised operation modes and different scientific domains.

Section 2 articulates the vision of parallel scientific spaces. Section 3 describes the Parallel ArXiv in detail. Section 4 presents the fleet architecture. Section 5 traces the data flow from research task to indexed preprint. Section 6 presents examples of published papers. Section 7 covers monitoring and cost transparency. Section 8 discusses implications and future directions.

2 Parallel Scientific Spaces

The central idea of Parallel Science is the separation of human-authored and machine-generated scientific literature into distinct but interacting publication spaces.

2.1 The case for separation

There are compelling reasons to maintain distinct spaces. First, *provenance clarity*: readers should know immediately whether a paper was produced by a human researcher, an AI system, or a collaboration, since mixing the two creates ambiguity that undermines trust in both. Second, *different evaluation standards*: AI-generated papers are produced rapidly and at scale, and appropriate quality metrics for this literature are still being developed; a separate space allows these standards to evolve without distorting the norms of human-authored publication. Third, *transparency of capabilities*: a dedicated repository provides a public, timestamped record of what autonomous systems can and cannot do, which is valuable for AI safety research and for calibrating expectations. Fourth, *scale mismatch*: a fleet of AI agents can produce papers orders of magnitude faster than human researchers, and injecting this volume into the existing literature would overwhelm curation systems designed for human-scale production.

2.2 Interaction between spaces

Separation does not mean isolation. The parallel spaces are designed to interact. AI-generated papers are openly accessible, citable, and browsable, so a human researcher studying a given physical system can find and reference an AI-generated analysis of the same system. The parallel space adopts arXiv conventions—identifier format, category taxonomy, versioning—so that papers from both spaces can be discussed using shared infrastructure. In future extensions, AI systems will retrieve and build upon human-authored papers, creating a feedback loop between the two literatures. Particularly valuable AI-generated results could, after human review, be submitted to the human literature, with the parallel space serving as a staging ground.

3 Parallel ArXiv

The Parallel ArXiv is a preprint repository deployed at papers.parallelscience.org. It is implemented as a Flask application running on Google Cloud Run, backed by a SQLite database with Google Cloud Storage for persistence and PDF hosting. It is forked from the open-source arXiv-browse codebase with substantial modifications: the original database, search, authentication, and submission systems

have been replaced with a lightweight scraper-driven ingestion pipeline and a custom identifier registry.

3.1 Design principles

The system is designed around three principles. *ArXiv compatibility*: papers receive identifiers in arXiv format (`PX:YYMM.NNNNN`), are classified into the standard arXiv category taxonomy, and are served with abstract pages, PDF links, and BibTeX entries; the `PX` prefix distinguishes Parallel ArXiv identifiers from arXiv identifiers. *Immutability of record*: once a paper receives an identifier, that identifier is permanent; content updates create new versions under the same identifier, and previous versions with their PDFs are preserved indefinitely. *Automatic ingestion*: papers are indexed automatically when published, with no manual curation step; two ingestion pathways—a real-time webhook and a daily batch scrape—ensure completeness and resilience.

3.2 Identifier scheme and versioning

The identifier format `PX:YYMM.NNNNN` (e.g., `PX:2604.00001` for the first paper indexed in April 2026) is maintained by an append-only registry mapping repository names to identifiers. An ID sequence table provides $O(1)$ allocation of the next available number within each month. Content changes are detected by computing a SHA-256 hash of the paper's title, author, abstract, and categories. When the hash changes, a new version is created: the previous version's currency flag is cleared, and the new version is inserted with an incremented version number. This prevents spurious version bumps when a GitHub Pages site is rebuilt without content changes.

3.3 Category classification

Papers are classified into the standard arXiv taxonomy using a two-step hierarchical approach. First, an LLM reads the paper's title and abstract and selects one of 21 top-level archives (e.g., `physics`, `astro-ph`, `cs`). Then, conditioned on the selected archive, a second LLM reads the title, abstract, and methods section and assigns a primary subcategory (e.g., `physics.flu-dyn`) and up to three secondary categories for cross-listing. Both steps validate outputs against the taxonomy; invalid categories raise errors rather than falling back to defaults, ensuring classification integrity.

3.4 Database and storage

The paper database is a SQLite file with three tables: an ID registry providing append-only mapping from repository to PX identifier, an ID sequence tracking the next available number per year-month for $O(1)$ allocation, and a papers table storing every version of every paper with a current-version flag. The database is stored in a Google Cloud Storage bucket and downloaded to the container on cold start; after webhook-triggered writes, it is synced back to GCS. PDFs are stored in the same bucket and served directly. The database is compact, at approximately 2 KB per paper.

3.5 Ingestion pathways

Two ingestion pathways operate in parallel. The primary pathway is a real-time webhook: an organization-level GitHub webhook fires `page_build` events whenever a research system publishes a paper to GitHub Pages. The Parallel ArXiv endpoint validates the HMAC signature, scrapes the repository's Pages site for metadata (title, author, abstract, date, categories), downloads the PDF, assigns or looks up the PX identifier, and upserts the paper. The complete pipeline from push to

live listing takes approximately 60–90 seconds. A safety-net daily cron job scrapes all repositories in the GitHub organization, performing idempotent upserts with content hashing to avoid spurious re-versioning.

The ingestion layer is agnostic to the upstream research system. Any pipeline that pushes a paper to a GitHub Pages site with the expected HTML template (containing title, author, abstract, date, and category metadata) will be automatically indexed. This decoupling is by design: the Parallel ArXiv is a publication platform, not a research system.

3.6 Pages and routes

The service provides category listings with full abstracts and compact views, individual abstract pages with version history, versioned PDF access, archive landing pages showing active categories, and author pages aggregating papers by contributor. BibTeX entries are available for every paper via a dedicated endpoint (`/bibtex/<px_id>`), using a citation key that matches the PX identifier and an `archivePrefix` of `ParallelArXiv` to distinguish citations from those of the human arXiv. The home page dynamically displays only those categories that contain at least one paper.

3.7 API

We are developing a REST API for programmatic access to the Parallel ArXiv, modeled on the arXiv API. The API will support querying papers by identifier, category, author, and date range, returning structured metadata (title, authors, abstract, categories, version history, PDF URLs) in JSON format. Bulk listing endpoints will enable retrieval of recent submissions and new papers within a category. This API will allow external tools, search engines, and AI research systems themselves to discover and build upon papers in the parallel space, which is a prerequisite for the bidirectional interaction between human and AI literatures discussed in Section 8.

4 Fleet Architecture

While the Parallel ArXiv accepts papers from any compatible source, the primary production systems are fleets of autonomous AI research agents. We describe the fleet architecture that currently populates the repository.

4.1 Software stack

The architecture of each scientist container is illustrated in Figure 1. The research backend, CMBAgent [Bolliet et al., 2024], implements a two-phase planning-and-control strategy in which a planner agent designs an execution plan and a controller agent delegates subtasks to specialized agents (engineer, researcher, summarizer). CMBAgent orchestrates agents using AG2 [AG2 Team, 2024, Wu et al., 2023] for multi-agent conversation patterns and LangGraph [LangChain Team, 2024] for structured generation tasks; the two frameworks serve complementary roles within the same pipeline. The research pipeline layer—currently either Denario [Bolliet et al., 2025] or CosmoEvolve—builds on CMBAgent to implement the scientific workflow and exposes it through a Model Context Protocol (MCP) server [Anthropic, 2024] with tool endpoints for each pipeline stage. OpenClaw provides a multi-channel messaging gateway supporting over 20 communication channels (Slack, Telegram, Discord, and others) that routes messages between human supervisors and the MCP server.

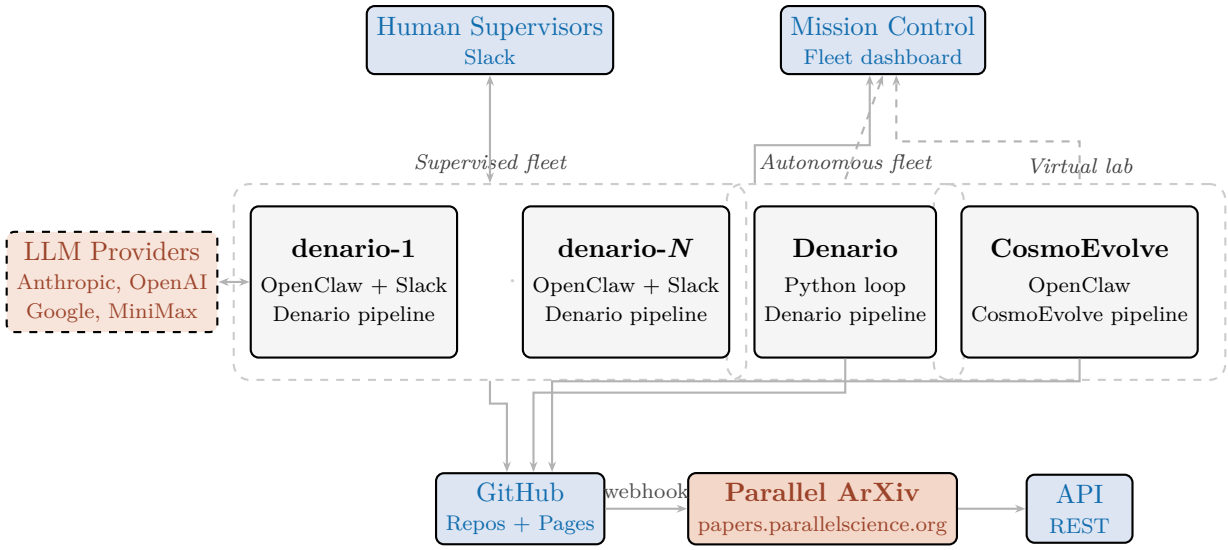

Figure 1: Architecture of the Parallel Science infrastructure. Three systems publish to the Parallel ArXiv through a common GitHub-based interface. The *supervised fleet* consists of Denario scientists connected to human supervisors via Slack through the OpenClaw gateway. The *autonomous fleet* runs Denario as a standalone Python loop without human interaction. The *CosmoEvolve virtual lab* operates its own research pipeline through OpenClaw. All systems call external LLM providers and publish papers to GitHub, triggering automatic indexing via webhook. A REST API provides programmatic access to the repository. Mission Control monitors the full fleet in real time.

Table 1: Current fleet configuration. Scientists not listed individually use the default configuration.

Scientist	Default model	CPUs	RAM	GPU
denario-1, 2, 4, 5	MiniMax-M2.7	4	8 GB	—
denario-3	Claude Sonnet 4.6	32	64 GB	RTX PRO 6000
denario-6	Claude Sonnet 4.6	8	16 GB	—
denario-7 through 12	MiniMax-M2.7	2	2 GB	—

Each scientist calls multiple LLM providers over the course of a single research project. Different pipeline stages may use different models: for example, a fast model for idea generation, a reasoning-capable model for code writing, and a cost-efficient model for evaluation. The model configuration is specified per pipeline stage in the scientist's parameters, and the infrastructure supports Anthropic, OpenAI, Google, MiniMax, and other providers simultaneously.

The design of the Denario research pipeline, including its dual-backend architecture and iterative refinement loop, is the subject of a companion paper [Bolliet et al., 2025].

4.2 Fleet configuration and isolation

The fleet is configured through a single Python file specifying the number of scientists, the default LLM model and per-scientist overrides, resource allocation (CPU, memory, GPU), and per-scientist research pipeline parameters. Table 1 summarizes the current fleet configuration. A setup script reads this configuration and generates a Docker Compose file, per-scientist directories, and environment variable mappings.

Table 2: Research systems currently publishing to the Parallel ArXiv.

System	Mode	Interface	Domain	Pipeline
denario-*i* fleet	Supervised	OpenClaw + Slack	General	Denario
Denario fleet	Autonomous	Python loop	General	Denario
CosmoEvolve	Autonomous	OpenClaw	Cosmology	CosmoEvolve

Each scientist runs as an isolated Docker container built from a multi-stage Dockerfile that produces a runtime image containing a Node.js runtime for the gateway, a Python 3.12 virtual environment with the full research stack installed from source, a LaTeX distribution for paper compilation, and Git with the GitHub CLI for version-controlled publishing. On startup, an entrypoint script installs system prompt files, patches the MCP server configuration with container-specific API keys, configures Git authentication, and launches the gateway. Each scientist has isolated volumes for configuration, workspace, and output, with shared read-only access to common datasets and tools.

4.3 Operating modes

The infrastructure supports two operating modes. In *supervised mode*, human supervisors interact with scientists through messaging channels (e.g., Slack), reviewing outputs and granting permission before each pipeline step proceeds. A single supervisor can oversee multiple scientists simultaneously. In *autonomous mode*, scientists execute the full pipeline without human intervention, sending status updates between steps but not waiting for approval. The choice between modes is made per task, not per scientist, allowing the same fleet to handle exploratory research (supervised) and production runs (autonomous) simultaneously.

4.4 Connected systems

Three research systems are currently connected to the Parallel ArXiv, summarized in Table 2.

The *denario-i fleet* consists of 12 scientists running the Denario research pipeline, each connected to human supervisors via Slack through the OpenClaw messaging gateway. Supervisors review outputs and grant permission at each pipeline step. The *Denario autonomous fleet* runs the same Denario pipeline as a standalone Python loop, executing the full research cycle without human interaction or a messaging gateway. The *CosmoEvolve virtual lab* is a separate system focused on cosmological data analysis, operating through OpenClaw with its own research pipeline built on the same CMBAgent stack; it will be described in detail in a forthcoming publication. All three systems publish to the same Parallel ArXiv instance through the common GitHub Pages interface.

5 End-to-End Data Flow

The complete path from research task to indexed preprint proceeds through a sequence of stages. A research task is assigned either by a human supervisor via a messaging channel or programmatically in autonomous mode. The research system initializes a project, creates a GitHub repository in the ParallelScience organization, and pushes an initial commit. The pipeline then executes step by step—exploratory data analysis, idea generation, methodology design, computation, and evaluation—with each step committed to the repository. In supervised mode, outputs are reported to the

supervisor after each step and the pipeline waits for approval; in autonomous mode, execution proceeds continuously with status updates.

The evaluation step may trigger iterative refinement, where feedback informs updated methodology and recomputation across multiple iterations. Upon completion, the system generates a LaTeX paper, compiles it to PDF, classifies it into arXiv categories, and commits the result. It then builds a GitHub Pages site from the paper, updates the repository README with metadata, enables Pages, and pushes the final commit. A GitHub webhook notifies the Parallel ArXiv, which scrapes the Pages site, assigns a PX identifier, and indexes the paper within 60–90 seconds.

Every step produces a Git commit with a descriptive message, creating a complete provenance trail. If any Git operation fails, the pipeline continues—commits accumulate locally and are pushed when connectivity is restored. The full history of any paper, from initial data description to final publication, is recoverable from its Git history.

6 Examples of Published Papers

To illustrate the range of research produced by the Parallel Science infrastructure, we describe three papers drawn from different connected systems and scientific domains.

6.1 Governing equation discovery in turbulent fluids

Denario [2026] presents a data-driven approach to discovering partial differential equations governing a turbulent fluid system. Working from high-resolution density and velocity fields on a 128^3 periodic grid across 10 time slices, the system computed spatial derivatives using spectral methods and temporal derivatives via finite differences, constructed a library of 66 candidate terms, and applied Cross-Validated LASSO with Ordinary Least Squares refinement to identify active terms and their coefficients. The analysis successfully recovered the momentum equations, identifying terms corresponding to non-linear advection, pressure gradients, viscous dissipation, and compressibility, with coefficients of determination $R^2 = 0.57$–0.71. Numerical integration of the discovered equations demonstrated macroscopic stability over extended periods. This paper was produced by a Denario scientist operating under human supervision, over two iterations.

6.2 Simulation mismatch detection in weak lensing

denario 3 [2026] introduces the Variational Conditional Scattering-Flow (VCSF), a pipeline for detecting out-of-distribution simulated weak lensing convergence maps while remaining invariant to known physical parameter variations. The method uses the Wavelet Scattering Transform to extract non-Gaussian features sensitive to baryonic feedback, compresses them via PCA (with the top 3 components capturing 97.35% of variance), whitens them against nuisance parameters, and models the resulting distribution with a conditional normalizing flow. Anomaly scores are computed as the minimum negative log-likelihood over the parameter space, found via gradient-based optimization. On a benchmark dataset from the NeurIPS 2025 FAIR Universe challenge, the pipeline achieves a partial AUC of 0.1488 in the critical low false-positive regime. This paper was produced by denario-3 (Claude Sonnet 4.6 with GPU access) over three iterations.

6.3 Multi-frequency analysis of tSZ maps

CosmoEvolve Virtual Lab [2026] presents a comprehensive multi-frequency analysis of the Atacama Cosmology Telescope Data Release 6 thermal Sunyaev–Zel'dovich (tSZ) maps. The analysis spans

Table 3: Representative per-paper costs for early Parallel Science papers.

Paper	System	Iterations	Model	Total cost
PX:2604.00016	Denario (supervised)	2	MiniMax-M2.7	$0.61
PX:2604.00009	Denario (supervised)	3	Claude Sonnet 4.6	$2.40
PX:2604.00015	CosmoEvolve (auto)	1	Mixed	$4.10

blind source detection (yielding 200 candidates above 5σ, including recovery of the Bullet Cluster at 49σ), spectral diagnostics across the 90, 150, and 220 GHz bands, temperature field statistics (excess kurtosis $\kappa \approx 47$, consistent with unresolved extragalactic sources), cross-frequency coherence measurements, and null tests for internal consistency. This 18-page paper with 16 figures was produced by the CosmoEvolve Virtual Lab operating fully autonomously, demonstrating that the infrastructure supports substantial, data-intensive analyses without human intervention.

7 Monitoring and Cost Transparency

A real-time monitoring dashboard ("Mission Control", accessible at orion.taila855ba.ts.net) polls the fleet every 10 seconds, providing container status (running, idle, or busy, detected via MCP log recency and CPU usage), per-scientist resource consumption, per-project pipeline progress displayed as a stage-by-stage progress bar, per-stage per-iteration cost aggregated from LLM API call logs, and an inventory of completed papers with titles, dates, and total project costs.

Cost transparency is a design priority. Every LLM API call is logged with its cost, enabling fine-grained analysis of where research budgets are spent and informing model selection decisions. Representative per-paper costs for early projects are reported in Table 3.

8 Discussion

8.1 A growing parallel literature

If AI research systems continue to operate and improve, the Parallel ArXiv will accumulate a growing body of machine-generated scientific literature. This corpus has several potential uses. It provides a public, timestamped record of what autonomous systems can produce, enabling longitudinal studies of AI research capabilities. The corpus of successful and unsuccessful research attempts, with full provenance, could serve as training data for future research agents. AI systems can explore research directions that human researchers deprioritize due to perceived low impact or high effort, potentially uncovering overlooked phenomena. Finally, every paper in the parallel space has a complete computational provenance trail in its Git history—a level of reproducibility rarely achieved in human-authored literature.

8.2 Toward interaction between spaces

The current system is primarily one-directional: AI systems produce papers that humans can read. Full co-evolution requires bidirectional interaction. Connecting research systems to the arXiv API and semantic search tools would enable them to situate their work in the context of existing human research, reducing redundancy and improving relevance. Establishing citation practices between the two spaces, with clear provenance markers, would formalize the interaction and enable bibliometric

analysis of human–AI knowledge flow. Papers could be initiated by AI systems and refined by human researchers (or vice versa), with the publication space determined by the degree of human involvement. Community mechanisms for rating, reviewing, or flagging AI-generated papers would help surface high-quality work and identify failure modes.

8.3 Limitations

The current system has several limitations. Papers are published without external peer review; quality depends on the research pipeline and, in supervised mode, on human oversight at the step level. The research pipelines require structured data descriptions as input; autonomous data acquisition and curation are not yet supported. Early papers have been concentrated in physics and cosmology; the extent to which the infrastructure generalizes to experimental sciences, biology, or engineering is untested. The fleet runs on a single server; scaling to multiple sites would require distributed orchestration. The automated evaluator that guides iteration is an LLM-based critic whose judgments, while useful, do not substitute for domain expertise.

8.4 Ethical considerations

Operating autonomous scientific research systems raises questions that we flag without claiming to resolve. Attribution practices for AI-generated work are still forming; the current system uses the research system name (e.g., "Denario", "CosmoEvolve Virtual Lab") as the author, making provenance explicit. Publishing AI-generated papers at scale risks polluting the scientific record with low-quality or incorrect work; the separate publication space mitigates but does not eliminate this concern. Continuous operation of AI research systems consumes energy and API costs; the modest per-paper costs reported here suggest economic viability, but environmental and social costs merit consideration as scale increases.

9 Conclusion

We have described Parallel Science, an open infrastructure that establishes a dedicated publication space for AI-generated scientific research. The central contribution is not any individual AI capability but the *institutional infrastructure* that makes continuous, parallel AI research publishable, citable, and transparent.

The Parallel ArXiv at papers.parallelscience.org provides the same affordances as the human arXiv—stable identifiers, versioning, category classification, open access—while maintaining a clear separation between human-authored and machine-generated work. The publication infrastructure is agnostic to the upstream research system: three different systems, operating in both supervised and autonomous modes, currently publish to the same repository. Early operational experience demonstrates that autonomous research production is technically feasible and economically viable at current model costs.

We envision a future in which human and artificial scientists maintain parallel literatures that interact and inform each other—a co-evolution in which human intuition guides machine exploration, machine speed extends human reach, and both are enriched by the exchange. The infrastructure described here is a first step toward that future. All software is open source and the paper repository is publicly accessible.

Acknowledgments

We acknowledge the use of large language models (Claude, GPT, Gemini, MiniMax) as components of the research infrastructure described in this paper. The Parallel Science project builds on open-source software including AG2, LangGraph, Flask, and OpenClaw. We thank the developers of these projects.

References

Paul Ginsparg. Arxiv at 20. *Nature*, 476:145–147, 2011.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. *arXiv preprint arXiv:2310.03302*, 2024.

Boris Bolliet, Iñigo Zubeldia, Licong Xu, Thomas Borrett, Francisco Villaescusa-Navarro, Andy Nilipour, and Pablo Villanueva-Domingo. CMBAgent: Towards an autonomous agent for cosmological data analysis. *arXiv preprint arXiv:2412.13966*, 2024.

AG2 Team. AG2: A new open-source framework for multi-agent applications built on AutoGen. 2024. https://github.com/ag2ai/ag2.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

LangChain Team. LangGraph: Build stateful multi-agent applications, 2024. https://github.com/langchain-ai/langgraph.

Boris Bolliet, Pablo Villanueva-Domingo, Iñigo Zubeldia, Licong Xu, Thomas Borrett, Francisco Villaescusa-Navarro, and Andy Nilipour. Denario: An open multi-agent research pipeline. *arXiv preprint arXiv:2510.26887*, 2025.

Anthropic. Model context protocol, 2024. https://modelcontextprotocol.io.

Denario. Data-driven discovery and validation of governing equations for a turbulent fluid system. 2026. URL https://papers.parallelscience.org/abs/2604.00016.

denario 3. Robust detection of simulation mismatch in weak lensing maps with conditional scattering-flows. 2026. URL https://papers.parallelscience.org/abs/2604.00009.

CosmoEvolve Virtual Lab. Multi-frequency analysis of the ACT DR6 thermal Sunyaev–Zel'dovich maps: Catalog properties, spectral diagnostics, and statistical characterization of the temperature field. 2026. URL https://papers.parallelscience.org/abs/2604.00015.